

تقرير

حول موضوع

Azure Storage Account

Cloud Storage · Security · Redundancy · Hands-on Lab

إعداد: مشعل فارس عبدالمالك عبده

ادهم صادق الممنصوب

محمد عبدالرقيب هزاع

إشراف الدكتور: احمد الشعراي

العام الدراسي ٢٠٢٦

فهرس المحتويات

- ١. مقدمة
- ٢. الجانب النظري
- ٣. الجانب العملي (خطوات التنفيذ في Azure Portal)
- ٤. السيناريو التطبيقي المتكامل
- ٥. الجداول والمقارنات
- ٦. الأخطاء والمشاكل الشائعة وطرق حلها
- ٧. أفضل الممارسات (Best Practices)
- ٨. أسئلة وأجوبة متوقعة للمناقشة
- ٩. الخاتمة
- ١٠. قائمة الأشكال والصور
- ١١. المراجع

مقدمة

يشهد العالم اليوم تحولًا متسارعًا نحو الحوسبة السحابية (Cloud Computing)، حيث أصبحت المؤسسات والشركات بمختلف أحجامها تعتمد على مزودي الخدمات السحابية لتخزين بياناتها وتشغيل تطبيقاتها بدلًا من الاعتماد الكامل على البنية التحتية التقليدية (On-Premises). ومن أبرز مزودي هذه الخدمات منصة Microsoft Azure، التي توفر مجموعة واسعة من الخدمات، من أهمها خدمة Azure Storage Account، والتي تُعد حجر الأساس لتخزين البيانات بمختلف أنواعها داخل بيئة Azure.

يهدف هذا التقرير إلى تقديم شرح أكاديمي شامل ومفصل لمفهوم Azure Storage Account، بدءًا من الجانب النظري الذي يوضح تعريفه، مكوناته، أنواع الخدمات المرتبطة به، مفاهيم الأمان والتكلفة، وصولًا إلى الجانب العملي الذي يوضح خطوات إنشائه واستخدامه الفعلي داخل بوابة Azure Portal، مع تطبيق عملي متكامل يحاكي سيناريو حقيقي لتطبيق يحتاج إلى تخزين أنواع مختلفة من البيانات.

وقد رُوعي في إعداد هذا التقرير الاعتماد على المفاهيم الرسمية الصادرة عن Microsoft قدر الإمكان، مع تبسيط الشرح ليكون مناسبًا لطالب مبتدئ في هذا المجال، وربط الجانب النظري بالجانب العملي بشكل مباشر لتسهيل الفهم والتطبيق.

أولاً: الجانب النظري

١. ما هو Azure Storage Account؟

◆ تعريف Azure Storage Account

حساب التخزين Storage Account هو الحاوية الرئيسية (الكيان الإداري) في منصة Microsoft Azure التي تحتوي على جميع كائنات البيانات التي يمكن تخزينها في خدمات Azure Storage. بمعنى آخر، هو المساحة السحابية التي تُمنح لك عند إنشائها، ومن خلالها يمكنك الوصول إلى جميع خدمات التخزين الفرعية (Blob و Files و Queue و Table).

◆ وظيفته ولماذا نحتاج إليه

يعمل Storage Account كنقطة دخول موحدة (Unique Namespace) يمكن الوصول إليها عبر HTTP أو HTTPS من أي مكان في العالم. تكمن الحاجة إليه في أن أي بيانات تحتاج إلى تخزين في Azure — سواء كانت ملفات، صوراً، نسخاً احتياطية، أو رسائل بين الخدمات — يجب أن تُخزن داخل حساب تخزين واحد على الأقل، لأن Azure لا يسمح بوجود بيانات (Blob أو File أو Queue أو Table) بمعزل عن حساب تخزين يحتضنها.

◆ كيف يعمل بشكل عام

عند إنشاء Storage Account، يمنحك Azure عنواناً فريداً (Endpoint) لكل نوع من خدماته المرتبطة بهذا الحساب، على سبيل المثال: <https://accountname.blob.core.windows.net> لخدمة Blob، و <https://accountname.file.core.windows.net> لخدمة Files. هذا يعني أن اسم الحساب يجب أن يكون فريداً عالمياً (Globally Unique) داخل نطاق Azure بأكمله.

◆ العلاقة بين Storage Account وخدمات Azure Storage

يُعد Storage Account بمثابة الغلاف الخارجي أو (المظلة الإدارية) التي تضم أربع خدمات تخزين رئيسية: Blob Storage، و Azure Files، و Queue Storage، و Table Storage. كل خدمة من هذه الخدمات مصممة لنوع معين من البيانات وطريقة استخدام معينة، لكنها جميعاً تشترك في نفس إعدادات الحساب مثل منطقة النشر (Region) ومستوى التكرار (Redundancy) ومستوى الأداء (Performance).

◆ مكونات Storage Account الأساسية

اسم الحساب (Account Name): معرف فريد عالمياً.

الاشتراك (Subscription): الحساب المالي المرتبط بالخدمة في Azure.

مجموعة الموارد (Resource Group): الحاوية المنطقية التي يوضع بداخلها الحساب.

المنطقة (Region): الموقع الجغرافي لمراكز بيانات Azure التي سيُستضاف عليها الحساب.

مستوى الأداء (Performance Tier): Standard أو Premium.

[نوع التكرار (Redundancy): يحدد عدد نسخ البيانات وأماكن تخزينها.

الخدمات الفرعية: Blob و Files و Queue و Table.

◆ أمثلة على استخداماته في الأنظمة والتطبيقات

من الأمثلة الواقعية: تطبيق تجارة إلكترونية يخزن صور المنتجات في Blob Storage، وموقع شركة يستخدم Azure Files لمشاركة ملفات الموظفين، ونظام معالجة طلبات يستخدم Queue Storage لتنسيق العمليات بين الخدمات المصغرة (Microservices)، وتطبيق إنترنت الأشياء (IoT) يخزن بيانات المستشعرات في Table Storage.

٢. لماذا نستخدم Azure Storage Account؟

تتعدد أسباب استخدام Azure Storage Account، ويمكن تلخيصها في النقاط التالية:

- تخزين البيانات: يوفر مساحة تخزين مرنة تتوسع تلقائيًا حسب الحاجة دون التقيد بسعة قرص صلب محدد.
- تخزين الملفات: إمكانية تخزين أي نوع من الملفات (مستندات، أرشيفات، ملفات تكوين) والوصول إليها من أي مكان.
- تخزين الصور والفيديوهات: مناسب جدًا لتخزين الوسائط الثقيلة نظرًا لقدرته على التعامل مع كائنات كبيرة الحجم عبر Blob Storage.
- تخزين النسخ الاحتياطية (Backups): يوفر خيارات تكرار متعددة تحمي البيانات من فقدان عند حدوث أعطال.
- تخزين بيانات التطبيقات: مثل ملفات السجلات (Logs) وبيانات الجلسات وملفات الإعدادات.
- الوصول إلى البيانات عبر الإنترنت: كل كائن مخزن يحصل على عنوان URL يمكن الوصول إليه عبر HTTPS من أي جهاز متصل بالإنترنت.
- قابلية التوسع (Scalability): يمكن أن يستوعب الحساب الواحد كميات هائلة من البيانات (بترابايت) دون الحاجة لأي تدخل يدوي من المستخدم.
- التوفر والاعتمادية (Availability & Reliability): بفضل آليات التكرار (Redundancy)، تضمن Microsoft نسبة توفر عالية جدًا للبيانات (تصل إلى ٩٩,٩٪ فأكثر حسب نوع التكرار).
- الأمان (Security): يدعم التشفير التلقائي للبيانات، والتحكم الدقيق في الصلاحيات، وطرق مصادقة متعددة.

٣. ما أنواع البيانات التي يمكن تخزينها؟

تختلف أنواع البيانات المناسبة لكل خدمة تخزين حسب طبيعة الخدمة وبنيتها:

- بيانات غير منظمة (Unstructured Data) مثل الصور والفيديوهات والمستندات ← تُخزن في Blob Storage.
- ملفات مشتركة يحتاج عدة مستخدمين أو أجهزة للوصول إليها بنفس الطريقة التي يُصل بها إلى مجلد على الشبكة ← تُخزن في Azure Files.
- رسائل قصيرة يتم تبادلها بين مكونات النظام لتنسيق المهام ← تُخزن في Queue Storage.
- بيانات شبه منظمة (Semi-structured Data) على شكل أزواج مفتاح-قيمة بدون علاقات معقدة بين الجداول ← تُخزن في Table Storage.
- مثال عملي: تطبيق توصيل طلبات يخزن صور الأصناف في Blob Storage، ويستخدم Queue Storage لإرسال إشعار لكل طلب جديد إلى نظام المطبخ، ويستخدم Table Storage لتخزين سجل الطلبات السابقة بسرعة استرجاع عالية.

٤. أنواع خدمات Azure Storage

◆ A. Blob Storage

خدمة Blob Storage (اختصارًا لـ Binary Large Object) هي الخدمة الأكثر استخدامًا في Azure Storage، وهي مخصصة لتخزين كميات كبيرة من البيانات غير المنظمة مثل الصور والفيديوهات والمستندات والنسخ الاحتياطية وملفات السجلات.

ما هو Blob؟ الـ Blob هو ببساطة أي ملف يتم رفعه إلى Azure Storage — بغض النظر عن نوعه أو امتداده — بمجرد رفعه فإنه يُعامل داخل النظام ككائن Blob له خصائص (Metadata) وصلاحيات وصول خاصة به.

أنواع Blobs الأساسية:

Block Blob: الأكثر شيوعًا، مخصص للملفات النصية والوسائط والمستندات، ويُقسم الملف إلى كتل (Blocks) يمكن رفعها بالتوازي.

Append Blob: مخصص للبيانات التي تُضاف باستمرار في نهاية الملف، مثل ملفات السجلات (Log Files).

Page Blob: مخصص للملفات التي تحتاج إلى قراءة/كتابة عشوائية بكفاءة عالية، مثل أقراص الأجهزة الافتراضية (VM Disks).

الفرق بين Blob وملف تقليدي: الملف التقليدي على جهاز الحاسوب يُخزن ضمن نظام ملفات هرمي (مجلدات فرعية حقيقية)، بينما الـ Blob يُخزن بشكل مسطح (Flat Namespace) داخل حاوية (Container)، وأي مسارات تظهر في اسم الـ Blob (مثل images/2026/photo.jpg) هي جزء من اسم الكائن نفسه وليست مجلدات حقيقية، وإن كانت واجهة Azure Portal تعرضها كأنها مجلدات لتسهيل التصفح.

مفهوم Container وعلاقته بـ Blob: الـ Container هو وحدة تنظيمية تشبه المجلد الرئيسي، ولا يمكن لأي Blob أن يوجد إلا داخل Container، ولا يمكن لأي Container أن يوجد إلا داخل Storage Account.

B. Azure Files

خدمة Azure Files توفر مساحات تخزين على شكل مشاركات ملفات (File Shares) يمكن الوصول إليها عبر بروتوكولات الشبكة القياسية مثل SMB و NFS، تمامًا كما لو كانت مجلد شبكة تقليديًا (Network Drive).

فائدتها الأساسية: تمكين عدة مستخدمين أو أجهزة افتراضية من الوصول المتزامن لنفس الملفات، مع إمكانية ربطها كمحرك أقراص (Mapped Drive) على أنظمة Windows أو Linux.

الفرق عن Blob Storage: بينما يعتمد Blob Storage على الوصول عبر REST API أو SDK فقط، تدعم Azure Files الوصول المباشر عبر بروتوكولات الشبكة التقليدية، كما أنها تحافظ على بنية مجلدات حقيقية (Hierarchical Namespace) بخلاف الطبيعة المسطحة لـ Blob.

مفهوم File Share: هو المساحة المخصصة داخل خدمة Azure Files، ويُشبه إلى حد كبير مجلدًا مشتركًا على خادم شركة.

مثال عملي: شركة تريد أن يصل جميع موظفيها إلى نفس مجلد المستندات الداخلية من أي مكان، فتقوم بإنشاء File Share وربطه كمحرك أقراص على أجهزتهم.

C. Queue Storage

خدمة Queue Storage مخصصة لتخزين وتبادل رسائل نصية صغيرة بين مكونات النظام المختلفة بشكل غير متزامن (Asynchronous).

لماذا تُستخدم؟ تُستخدم لفصل مكونات النظام عن بعضها (Decoupling)، بحيث يمكن لمكون ما إرسال رسالة إلى الطابور دون الحاجة لانتظار استجابة فورية من المكون المستقبل، مما يزيد من مرونة النظام وقدرته على التوسع.

مفهوم الرسائل والطوابير: الرسالة (Message) هي وحدة بيانات صغيرة (نص أو JSON) يتم وضعها في طابور (Queue)، وتُعالج الرسائل بترتيب الوصول (FIFO تقريبًا)، حيث تقوم خدمة أو تطبيق آخر بسحب الرسالة ومعالجتها ثم حذفها من الطابور.

مثال عملي: عند رفع مستخدم فيديو إلى منصة مشاركة فيديوهات، يمكن وضع رسالة في Queue Storage تشير إلى ضرورة معالجة الفيديو (ضغط، تحويل صيغة)، وتقوم خدمة منفصلة بسحب هذه الرسالة وتنفيذ المعالجة في الخلفية دون أن ينتظر المستخدم.

D. Table Storage

خدمة Table Storage هي قاعدة بيانات NoSQL بسيطة مخصصة لتخزين كميات كبيرة من البيانات شبه المنظمة بسرعة استرجاع عالية وتكلفة منخفضة.

نوع البيانات: بيانات على شكل أزواج (Key-Value) موزعة على أعمدة (Properties) داخل صفوف تُسمى Entities، دون فرض بنية جدول ثابتة كما في قواعد البيانات العلائقية.

مفهوم NoSQL: يعني عدم الاعتماد على العلاقات الجدولية التقليدية (Joins) ولا على مخطط ثابت (Fixed Schema)، بحيث يمكن أن يحتوي كل صف على أعمدة مختلفة عن الصف الآخر.

متى نستخدمها؟ عندما تحتاج إلى تخزين كميات ضخمة من البيانات البسيطة (مثل سجلات المستخدمين أو بيانات الأجهزة) مع الحاجة إلى استعلامات سريعة عبر مفتاح أساسي، دون الحاجة لعلاقات معقدة بين الجداول.

مثال عملي: تطبيق إنترنت الأشياء يخزن قراءات آلاف المستشعرات كل دقيقة؛ كل قراءة تُخزن كـ Entity منفصلة تحتوي على معرف الجهاز والوقت والقيمة.

◆ جدول مقارنة بين خدمات Azure Storage الأربع

المعيار	Blob Storage	Azure Files	Queue Storage	Table Storage
الوظيفة	تخزين كائنات ثنائية كبيرة	مشاركة ملفات عبر الشبكة	تبادل رسائل بين الخدمات	تخزين بيانات NoSQL
نوع البيانات	بيانات غير منظمة	ملفات ومجلدات	رسائل نصية قصيرة	بيانات شبه منظمة
طريقة الاستخدام	REST API / SDK	/ SMB / NFS REST	REST API / SDK	/ REST API SDK
أمثلة	صور، فيديو، نسخ احتياطية	مجلد شركة مشترك	إشعارات معالجة الطلبات	سجلات أجهزة IoT
متى نستخدمها	بيانات ضخمة غير منظمة	وصول متعدد كمجلد شبكة	فصل مكونات النظام	استعلامات سريعة وبسيطة

٥. الفرق بين Storage Account و Blob Container

كثيرًا ما يخلط الطلاب المبتدئون بين هذين المفهومين، رغم أن الفرق بينهما بسيط جدًا إذا نظرنا إلى العلاقة الهرمية بينهما:

Azure Storage Account



Blob Container



Blob

Storage Account هو المستوى الأعلى: الحساب السحابي الكامل الذي يضم جميع الخدمات (Blob، Files، Queue، Table). أما Blob Container فهو مستوى تنظيمي أدنى موجود فقط داخل خدمة Blob Storage، ويُستخدم لتجميع مجموعة من الـ Blobs المرتبطة ببعضها منطقيًا (مثل تجميع كل صور المنتجات في container واحد).

هل يمكن أن يوجد Container بدون Storage Account؟ لا، هذا مستحيل تقنيًا؛ فكل Container يجب أن يكون تابعًا لحساب تخزين واحد فقط، ولا يمكن إنشاؤه بشكل مستقل.

وظيفة كل واحد: Storage Account مسؤول عن الإعدادات العامة (المنطقة، التكرار، الأمان العام، الفوترة)، بينما Container مسؤول عن تنظيم الملفات (Blobs) وتحديد مستوى الوصول العام لها (Public/Private) على مستوى المجموعة.

أين تُخزن الملفات فعليًا؟ الملفات (Blobs) تُخزن داخل الـ Container، والـ Container بدوره جزء من الـ Storage Account، وكل ذلك يُخزن فعليًا على البنية التحتية لمراكز بيانات Microsoft في المنطقة (Region) التي اخترتها.

مثال عملي: تخيل Storage Account كأنه "مبنى شركة"، وكل Container كأنه "قسم" داخل المبنى (قسم الصور، قسم المستندات)، والـ Blob هو "الملف" الموجود داخل ذلك القسم.

٦. مفهوم Redundancy (التكرار)

Redundancy يعني الاحتفاظ بأكثر من نسخة من نفس البيانات في أماكن مختلفة (سواء داخل نفس مركز البيانات أو في مناطق جغرافية مختلفة)، بهدف حماية البيانات من فقدان أو الانقطاع في حال حدوث عطل في الأجهزة أو كارثة طبيعية تصيب أحد مراكز البيانات.

لماذا نستخدمها؟ لأن الاعتماد على نسخة واحدة فقط من البيانات يجعلها عرضة للفقدان الكامل عند أي عطل، بينما وجود نسخ متعددة يضمن استمرارية توفر البيانات (High Availability) واستمرارية العمل (Business Continuity).

كيف تحمي البيانات؟ يقوم Azure تلقائيًا بنسخ البيانات إلى الأماكن الإضافية المحددة في نوع التكرار المختار، دون أي تدخل يدوي من المستخدم. وعند حدوث عطل في إحدى النسخ، يقوم Azure تلقائيًا بالتحويل إلى نسخة أخرى سليمة.

◆ أنواع Redundancy الرئيسية

- LRS (Locally Redundant Storage): نسخ البيانات ٣ مرات داخل نفس مركز البيانات (نفس المنطقة الفعلية).
- ZRS (Zone-Redundant Storage): نسخ البيانات عبر ٣ مناطق توافر (Availability Zones) مختلفة داخل نفس الـ Region.
- GRS (Geo-Redundant Storage): نسخ البيانات محليًا (LRS) ثم نسخها بشكل غير متزامن إلى منطقة جغرافية ثانوية بعيدة.
- GZRS (Geo-Zone-Redundant Storage): دمج بين ZRS في المنطقة الأساسية ونسخ جغرافي إلى منطقة ثانوية.

◆ جدول مقارنة بين أنواع Redundancy

متى نستخدمه	التكلفة	الحماية من	عدد النسخ	النوع
بيانات غير حساسة أو تجريبية	الأقل تكلفة	عطل قرص أو خادم	٣ نسخ داخل نفس المركز	LRS
تطبيقات تتطلب توفرًا أعلى محليًا	متوسطة	عطل مركز بيانات كامل	٣ نسخ عبر مناطق توافر	ZRS
بيانات مهمة تحتاج حماية جغرافية	أعلى من ZRS	كارثة إقليمية كاملة	٦ نسخ (محلي + منطقة بعيدة)	GRS
أنظمة حساسة تتطلب أعلى مستوى حماية	الأعلى تكلفة	عطل منطقة توافر و كارثة إقليمية معًا	٦ نسخ (ZRS + منطقة بعيدة)	GZRS

تأثيرها على التكلفة والتوفر: كلما زاد عدد النسخ ومدى انتشارها الجغرافي، زادت نسبة التوفر المضمونة (SLA) وزادت التكلفة الشهرية بالمقابل، لذلك يجب اختيار نوع التكرار بناءً على أهمية البيانات وليس تلقائيًا اختيار الأعلى دائمًا.

٧. الفرق بين Public Access و Private Access

Public Access يعني السماح لأي شخص في العالم بالوصول إلى الملف عبر رابط URL مباشر دون الحاجة إلى أي مفتاح أو تصريح مسبق، بينما Private Access يعني منع أي وصول مباشر للملف إلا من خلال طرق مصادقة محددة مثل مفاتيح الوصول (Access Keys) أو توقيعات الوصول المشترك (SAS).

كيف يمكن الوصول إلى البيانات في كل حالة؟ في حالة Public، يكفي معرفة رابط الملف لفتحه مباشرة من المتصفح. أما في حالة Private، فإن محاولة فتح الرابط مباشرة تُظهر خطأ رفض الوصول (Access Denied ٤٠٣)، ويجب توليد رابط SAS مؤقت أو استخدام بيانات اعتماد صحيحة للوصول.

تحذير أمني: Public Access يعرض البيانات الحساسة لخطر الوصول من أي جهة غير مصرح لها، بما في ذلك محركات البحث وبرمجيات الزحف الآلي (Bots)، وهو سبب رئيسي في كثير من حوادث تسريب البيانات السحابية.

لماذا يُفضّل استخدام Private Access في البيانات الحساسة؟ لأنه يمنع أي وصول غير مصرح به من الأساس، ويتيح التحكم الدقيق في من يمكنه الوصول ولأي مدة زمنية عبر SAS، بدلاً من ترك الملف مكشوفًا بشكل دائم.

مثال عملي: موقع تجارة إلكترونية يمكن أن يجعل صور المنتجات Public لأنها مخصصة للعرض العام، لكنه يجب أن يجعل ملفات فواتير العملاء أو مستنداتهم الشخصية Private، ولا يسمح بالوصول إليها إلا عبر رابط SAS مؤقت يُؤد بعد تسجيل الدخول.

◆ أفضل الممارسات الأمنية

- جعل مستوى الوصول الافتراضي Private ما لم تكن هناك حاجة واضحة لجعله Public.
- استخدام SAS محدود المدة بدلاً من فتح الوصول العام بشكل دائم.
- مراجعة إعدادات Public Access على مستوى الحساب والـ Container بشكل دوري.

٨. أهم الاعتبارات الأمنية والتكلفة

◆ الجوانب الأمنية

- Authentication (المصادقة): التحقق من هوية الجهة التي تحاول الوصول إلى الحساب، عبر مفاتيح الوصول أو Microsoft Entra ID.
- Authorization (التفويض): تحديد ما يُسمح لهذه الجهة القيام به بعد التحقق من هويتها (قراءة فقط، كتابة، حذف...).
- Access Keys: مفاتيح سرية طويلة تمنح صلاحية كاملة على الحساب، ويجب حفظها بسرية تامة وعدم مشاركتها أو تضمينها في الكود المصدري مباشرة.
- Shared Access Signature (SAS): رابط أو توقيع مؤقت يمنح صلاحيات محددة (مثل القراءة فقط) لمدة زمنية محددة، وهو البديل الآمن لمشاركة Access Keys الكاملة.
- Microsoft Entra ID: نظام إدارة هوية مركزي (سابقاً Azure Active Directory) يتيح التحكم في صلاحيات المستخدمين والتطبيقات بشكل أدق وأكثر أماناً من الاعتماد فقط على المفاتيح.
- Encryption (التشفير): يقوم Azure بتشفير جميع البيانات تلقائياً أثناء التخزين (Encryption at Rest) باستخدام معايير قوية مثل AES-256.
- HTTPS: يُفضّل دائماً فرض النقل الآمن (Secure Transfer Required) بحيث لا يُسمح بالاتصال إلا عبر HTTPS المشفّر.
- Public Access: كما ذكر سابقاً، يجب تقييده على مستوى الحساب والـ Container عند عدم الحاجة إليه.
- Network Access: يمكن تقييد الوصول إلى شبكات افتراضية محددة (Virtual Networks) أو عناوين IP معينة فقط عبر جدار الحماية الخاص بالتخزين (Storage Firewall).

◆ عوامل التكلفة

- كمية البيانات المخزنة (بالجيجابايت/تيرابايت شهرياً).
- عدد عمليات القراءة والكتابة (Transactions).
- نوع طبقة التخزين: Hot (ساخنة، وصول متكرر) أو Cool (باردة، وصول أقل) أو Archive (أرشفية، وصول نادر جداً وأرخص تكلفة تخزين).
- نوع Redundancy المختار (كلما زادت النسخ، زادت التكلفة).
- نقل البيانات خارج منطقة Azure (Data Egress)، حيث تكون عمليات الإخراج عادة أكثر تكلفة من عمليات الإدخال.

◆ كيف يمكن تقليل التكلفة؟

- اختيار طبقة التخزين المناسبة لنمط الوصول الفعلي للبيانات (نقل البيانات النادرة الاستخدام إلى Cool أو Archive).
- اختيار مستوى Redundancy مناسب لأهمية البيانات وعدم المبالغة فيه (LRS بدل GZRS عند عدم الحاجة).
- تفعيل سياسات دورة حياة البيانات (Lifecycle Management) لنقل أو حذف البيانات القديمة تلقائياً.
- مراقبة الاستخدام دورياً عبر Azure Cost Management.

ثانيًا: الجانب العملي — التنفيذ خطوة بخطوة في Azure Portal

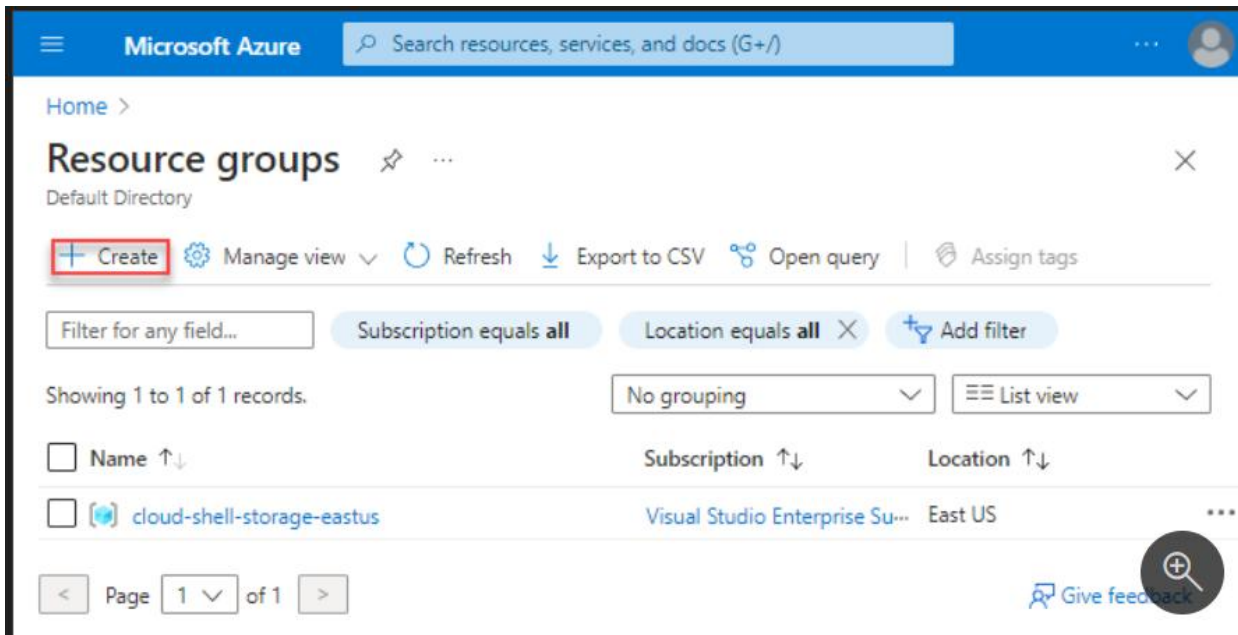
هذا القسم مكتوب ليكون قابلاً للتنفيذ الكامل من قبل طالب مبتدئ داخل Azure Portal دون الحاجة لأي مصدر خارجي، وقد أرفقت لقطات الشاشة الفعلية لكل خطوة من خطوات التنفيذ أدناه.

الخطوة ١: إنشاء Resource Group

سجل الدخول إلى <https://portal.azure.com> باستخدام حساب Microsoft أو حساب Azure for Students.
من القائمة الجانبية، اختر Resource groups، ثم اضغط على زر + Create.
Subscription: اختر الاشتراك الذي سيتم الفوترة عليه (مثل Azure for Students أو Free Trial).
Resource group name: أدخل اسمًا واضحًا مثل rg-storage-demo.
Region: اختر منطقة قريبة جغرافيًا لتقليل زمن الاستجابة، مثل West Europe.
لماذا نحتاج Resource Group؟ لأنه حاوية منطقية تجمع كل الموارد المرتبطة بمشروع واحد (Storage Account، وغيرها لاحقًا)، مما يسهل إدارتها ومراقبة تكلفتها وحذفها دفعة واحدة عند الانتهاء من المشروع.
اضغط Review + Create، ثم Create. بعد بضع ثوانٍ ستظهر رسالة Deployment succeeded، وتصبح مجموعة الموارد جاهزة.

الشكل (١)

إنشاء Resource Group



Home > Resource groups >

Create a resource group

Basics Tags Review + create

Resource group - A container that holds related resources for an Azure solution. The resource group can include all the resources for the solution, or only those resources that you want to manage as a group. You decide how you want to allocate resources to resource groups based on what makes the most sense for your organization. [Learn more](#)

Project details

Subscription *

Resource group *

Resource details

Region *

[Review + create](#) < Previous Next : Tags >

Home >

Resource groups

Default Directory

+ Create Manage view Refresh Export to CSV Open query Assign tags

Filter for any field... Subscription equals all Location equals all Add filter

Showing 1 to 2 of 2 records. No grouping List view

☐ Name ↑↓	Subscription ↑↓	Location ↑↓	
☐ cloud-shell-storage-eastus	Visual Studio Enterprise Su...	East US	...
☐ myResourceGroup0816	Visual Studio Enterprise Su...	Central US	...

< Page 1 of 1 >

Give feedback

Microsoft Azure Search resources, services, and docs (G+/)

Home >

Resource groups

Default Directory

+ Create Manage view

Filter for any field... Sub

Showing 1 to 2 of 2 records.

☐ Name ↑↓	Subscription ↑↓	Location ↑↓	
☐ cloud-shell-storage-eastus	Visual Studio Enterprise Su...	East US	...
☐ myResourceGroup0816	Visual Studio Enterprise Su...	Central US	...

< Page 1 of 1 >

Notifications

[More events in the activity log →](#)

✓ **Resource group created**

Creating resource group 'myResourceGroup0816' succeeded.

[Go to resource group](#) Pin

Cloud Shell Directories + subscriptions **Notifications** Settings Support & Troubleshooting Feedback

2 minutes ago

الخطوة ٢: إنشاء Storage Account

من شريط البحث العلوي، اكتب Storage accounts ثم اضغط + Create.

Subscription: نفس الاشتراك المستخدم سابقًا.

Resource Group: اختر rg-storage-demo التي أنشأناها في الخطوة السابقة.

Storage account name: اسم فريد عالميًا بحروف صغيرة وأرقام فقط بدون مسافات، مثل storedemo2026xy.

Region: نفس المنطقة التي اخترتها لمجموعة الموارد لتقليل التكلفة وزمن الاستجابة.

Performance: اختر Standard (مناسب لمعظم المشاريع التعليمية)؛ أما Premium فمخصص للأداء العالي جدًا وتكلفته أعلى.

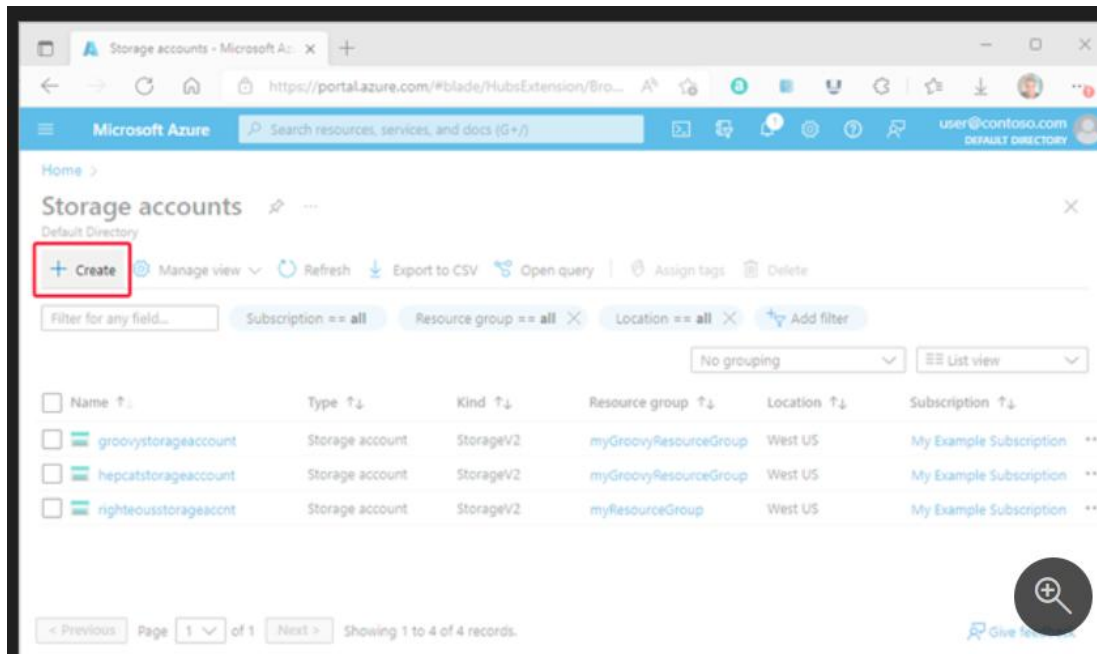
Redundancy: اختر LRS كخيار اقتصادي مناسب للمشاريع التعليمية والتجريبية، لأن الحماية الجغرافية العالية غير ضرورية في هذه المرحلة.

انتقل إلى تبويب Advanced وتأكد أن خيار Require secure transfer for REST API operations (Enabled) للحفاظ على الأمان.

اضغط Review + Create، تحقق من الملخص، ثم اضغط Create وانتظر حتى تكتمل عملية النشر (Deployment).

الشكل (٢)

إنشاء Storage Account وإعداداته



الخطوة ٣: إنشاء Blob Container

بعد اكتمال الإنشاء، اضغط Go to resource للدخول إلى صفحة الحساب.

من القائمة الجانبية، تحت Data storage، اختر Containers.

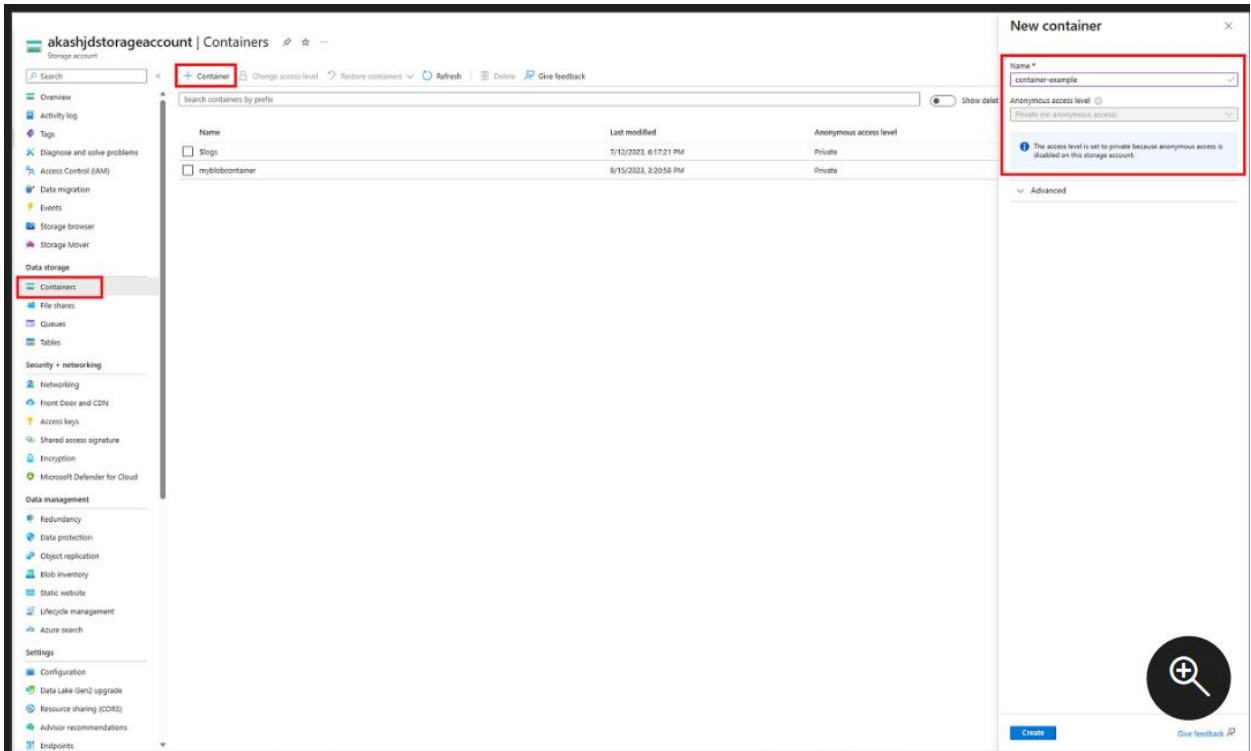
اضغط + Container، ثم أدخل اسمًا مناسبًا مثل images-container.

Public access level: اختر (Private (no anonymous access) في المشاريع التعليمية والبيانات غير المخصصة للعرض العام، لتفادي أي كشف غير مقصود للبيانات.

اضغط Create لإنشاء الـ Container.

الشكل (٣)

إنشاء Blob Container وتحديد مستوى الوصول



الخطوة ٤: رفع ملف إلى Container

ادخل إلى الـ Container الذي أنشأته بالضغط على اسمه.

اضغط زر Upload أعلى الصفحة.

اضغط على Browse for files واختر ملفًا من جهازك (مثل صورة أو ملف PDF).

اضغط Upload لبدء رفع الملف.

بمجرد اكتمال الرفع، سيظهر الملف داخل قائمة الـ Container، وقد أصبح الآن Blob رسميًا داخل خدمة Blob Storage.

توضيح العلاقة: الملف أصبح الآن يقع ضمن التسلسل Blob ← Container ← Storage Account كما شُرح سابقًا في الجانب النظري

رفع ملف إلى Blob Container  الشكل (٤):

Upload blob

sample-container/

Files

"upload-blob.png" 

☐ Overwrite if files already exist

^ Advanced

Authentication type

Azure AD user account Account key

Blob type

Block blob 

☒ Upload .vhd files as page blobs (recommended)

Block size

4 MB 

Access tier

Hot (Inferred) 

Upload to folder

virtual-folder

Blob index tags

Key	Value
-----	-------

Blob index tags ⓘ

Key

Value

Encryption scope

- ☒ Use existing default container scope
- ☐ Choose an existing scope

Retention policy

- ☒ No retention
- ☐ Choose custom retention period:

10/25/2021



1:35:12 PM

ⓘ Enable version-level immutability on the container to set a retention policy.

Upload

Dashboard > storagesamples >

sample-container ⓘ
Container

Search (Ctrl+/) <<

Upload Change access level Refresh Delete Change tier Acquire lease Break lease ...

Overview

Access Control (IAM)

Settings

Access policy

Properties

Metadata

Editor (preview)

Authentication method: Azure AD User Account (Switch to Access Key)

Location: sample-container

Search blobs by prefix (case-sensitive)

Show deleted blobs

	Name	Modified	Access tier	Blob type	Size
☐	level1				
☒	blob1.txt	11/27/2019, 1:53:15 P...	Hot (Inferred)	Block blob	48 B
☒	blob2.txt	3/11/2019, 10:18:04 P...	Hot (Inferred)	Block blob	20 B
☒	blob3.txt	2/25/2019, 11:59:30 ...	Hot (Inferred)	Block blob	9 B
☐	blob4.txt	2/25/2019, 12:01:16 P...	Hot (Inferred)	Block blob	9 B
☐	blob5.txt	2/25/2019, 12:01:39 P...	Hot (Inferred)	Block blob	9 B
☐	blob6.txt	2/25/2019, 12:06:56 P...	Hot (Inferred)	Block blob	9 B
☐	blob7.txt	2/25/2019, 12:06:56 P...	Hot (Inferred)	Block blob	9 B
☐	blob8.txt	2/25/2019, 12:06:55 P...	Hot (Inferred)	Block blob	9 B
☐	blob9.txt	2/25/2019, 12:06:55 P...	Hot (Inferred)	Block blob	9 B
☐	logfile.txt	9/19/2019, 4:06:49 PM	Hot (Inferred)	Block blob	13 B

الخطوة ٥: استعراض الملف وطريقة الوصول إليه

اضغط على اسم الملف المرفوع لعرض تفاصيله وخصائصه (Properties) مثل الحجم، تاريخ آخر تعديل، ونوع المحتوى.

في أعلى صفحة تفاصيل الملف يظهر حقل URL وهو الرابط الفريد لهذا الـ Blob.

إذا كان الـ Container عام (Public)، يمكن نسخ هذا الرابط ولصقه مباشرة في متصفح جديد وسيُفتح الملف بدون أي طلب تسجيل دخول.

إذا كان الـ Container خاص (Private) كما في مشروعنا، فإن فتح نفس الرابط مباشرة سيُظهر خطأ ٤٠٣ (Server failed to authenticate the request).

للسماح بالوصول الآمن رغم كون الحساب Private، نضغط بزر الفأرة الأيمن على الملف ونختار Generate SAS، نحدد الصلاحيات (قراءة فقط مثلاً) ومدة الصلاحية، ثم نولد رابطاً مؤقتاً يمكن مشاركته بأمان.

الشكل (٥)

خصائص الملف ورابط الوصول (URL) وتوليد SAS

The screenshot shows the 'blob1.txt' properties in the Azure Storage portal. The 'URL' field is highlighted with a red box. The 'CREATION TIME' is 2/25/2019, 2:59:30 PM. The 'SERVER ENCRYPTED' status is 'true'.

Property	Value
URL	https://storagesamples.blob.core.windows.net/sampl...
LAST MODIFIED	10/15/2019, 3:51:58 PM
CREATION TIME	2/25/2019, 2:59:30 PM
TYPE	Block blob
SIZE	21 B
ACCESS TIER	Hot (Inferred)
ACCESS TIER LAST MODIFIED	N/A
SERVER ENCRYPTED	true
ETAG	0x8D751A923E07042
CONTENT-TYPE	application/octet-stream
CONTENT-MD5	m91jBTStqgiwHi8TdHPing==
LEASE STATUS	Unlocked
LEASE STATE	Available
LEASE DURATION	-
COPY STATUS	-
COPY COMPLETION TIME	-

The screenshot shows the 'blobs.png' file in the Azure Storage portal. The 'Download' button is highlighted with a red box. The 'Authentication method' is 'Access key' and the 'Location' is 'sample-container'.

Name	Modified	Access tier	Blob type	Size
blobs.png	8/10/2020, 3:02:16 PM	Hot (Inferred)	Block blob	25.81 KiB

الخطوة ٦: إنشاء File Share

من القائمة الجانبية لنفس Storage Account، تحت Data storage، اختر File shares.

اضغط + File share، وأدخل اسمًا مثل company-docs.

حدد Tier (مستوى الأداء) واترك الإعداد الافتراضي Transaction optimized للاستخدام التعليمي.

اضغط Create.

ادخل إلى الـ File Share الذي أنشأته، ثم اضغط Upload واختر ملفًا من جهازك لرفعه، تمامًا كما فعلنا مع Blob Container.

الفرق بين File Share و Blob Container: الـ File Share يحافظ على بنية مجلدات حقيقية ويمكن ربطه كمحرك أقراص شبكي (Network Drive) عبر SMB، بينما الـ Container مسطح ويُستخدم فقط عبر REST API أو الواجهة.

متى نستخدم كل واحد؟ نستخدم Blob عندما تكون البيانات غير منظمة (صور، فيديو، نسخ احتياطية) ولا تحتاج وصولًا كمجلد شبكة، ونستخدم File Share عندما نحتاج محاكاة مجلد مشترك تقليدي يصل إليه عدة مستخدمين أو أجهزة في نفس الوقت.

الشكل (٦) إنشاء File Share ورفع ملف إليه

New file share

Basics Backup Review + create

The file share is billed by provisioned share size, IOPS and throughput, regardless of the used capacity. The minimum share size is 32 GiB. [Learn more](#)

Name *

Provisioned storage (GiB) *
[Set to maximum](#)

Performance

Provisioned IOPS and throughput * ☒ Recommended provisioning
☐ Manually specify IOPS and throughput

IOPS * 3000-102400

Throughput (MiB/sec) * 100-10340

Protocol ⓘ * ☒ SMB
☐ NFS

الخطوة ٧: استعراض إعدادات Redundancy و Access

للوصول إلى إعدادات Redundancy: من صفحة الحساب الرئيسية، اذهب إلى Data management ثم Redundancy، حيث يظهر النوع المختار حالياً (LRS في مشروعنا) مع إمكانية تغييره لاحقاً إن رغبت.

للتحقق من إعدادات Public/Private Access على مستوى الحساب: اذهب إلى Configuration ضمن Settings، وتحقق من قيمة Allow Blob public access.

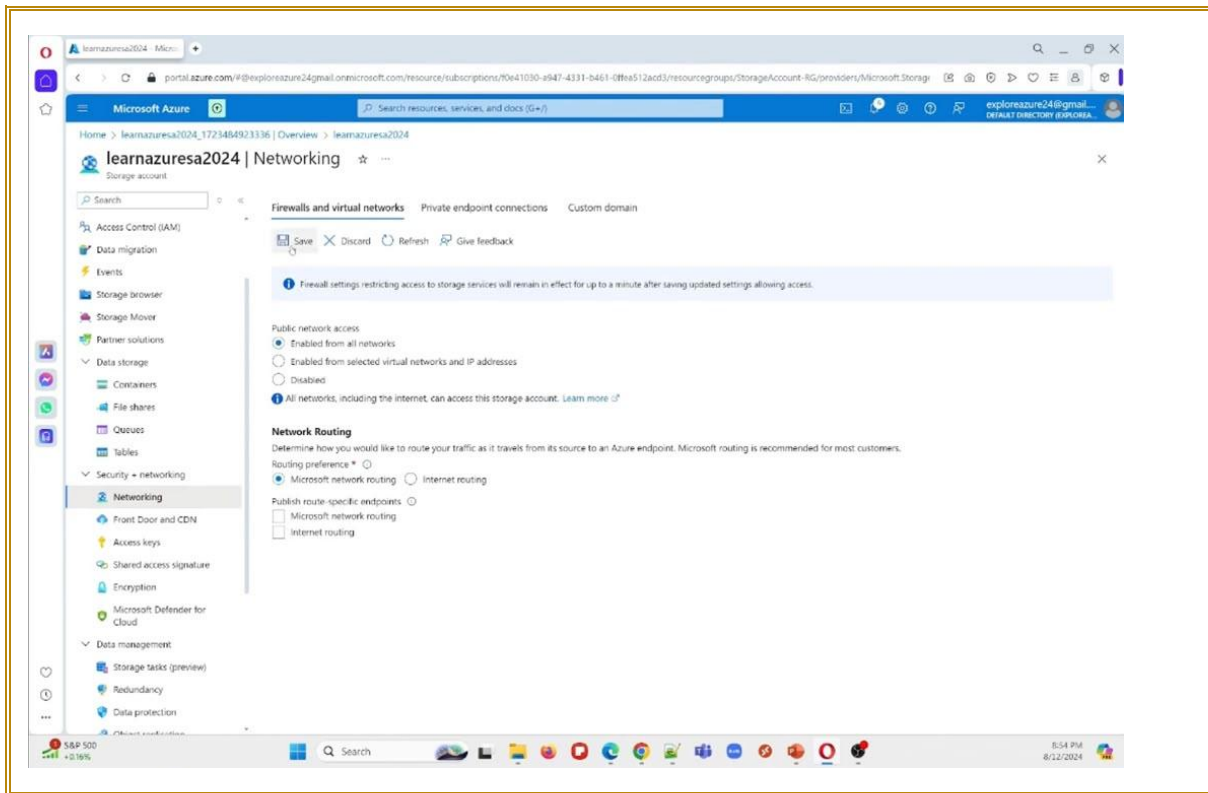
للتحقق من إعداد الوصول على مستوى Container معين: افتح الـ Container واضغط على Change access level لرؤية القيمة الحالية (Private / Blob / Container).

ما الذي يجب ملاحظته؟ التأكد الدوري من أن هذه الإعدادات لم تتغير عن طريق الخطأ، خصوصاً في البيئات الإنتاجية، لأن أي تغيير غير مقصود في مستوى الوصول قد يعرض البيانات للخطر.

الشكل (٧)

إعدادات Redundancy و Access داخل Storage Account

Home > samplesalrs2ragrs



ثالثًا: سيناريو عملي متكامل

لنفترض أننا نبني تطبيق ويب لإدارة طلبات خدمة عملاء، يحتاج إلى تخزين أربعة أنواع من البيانات: صور مرفقة من العملاء، ملفات PDF للتقارير، ملفات مشتركة يستخدمها فريق الدعم، ورسائل تنسيق بسيطة بين الخدمات.

السبب	الخدمة المناسبة	نوع البيانات
بيانات غير منظمة، حجم متغير، وصول عبر URL	Blob Storage (Block Blob)	صور مرفقة من العملاء
ملفات ثابتة يمكن أرشفتها لاحقًا في Cool/Archive	Blob Storage (Block Blob)	ملفات PDF للتقارير
وصول متعدد المستخدمين كمجلد شبكة عبر SMB	Azure Files	ملفات فريق الدعم المشتركة
معالجة غير متزامنة وفصل مكونات النظام	Queue Storage	رسائل تنسيق بين الخدمات

تنظيم البيانات: يُنشأ Storage Account واحد باسم واضح مثل stcustomersupportapp، وتحتهِ يُنشأ Container باسم customer-attachments لصور العملاء، وContainer آخر باسم reports-pdf للتقارير، بينما يُنشأ File Share باسم support-team-files لفريق الدعم، وQueue باسم ticket-notifications لرسائل التنسيق.

رفع الملفات والوصول إليها: تُرفع الصور والتقارير عبر التطبيق نفسه باستخدام SDK الخاص بـ Azure Storage، ويُؤد لكل عميل رابط SAS مؤقت للوصول إلى مرفقاته فقط، بينما يصل فريق الدعم إلى File Share مباشرة عبر أجهزتهم المرتبطة بالشبكة الداخلية.

الحماية: تُبقى جميع الـ Containers على وضع Private، ويُستخدم GRS بدلاً من LRS لأن بيانات العملاء تُعد بيانات مهمة تستحق الحماية الجغرافية، بينما يمكن الاكتفاء بـ LRS لملفات السجلات المؤقتة الأقل أهمية.

رابعًا: رسومات توضيحية للبنية

البنية الهرمية الكاملة لحساب التخزين



مسار الوصول إلى البيانات

User → Internet → Azure → Storage Account → Container → Blob

خامساً: الجداول والمقارنات المجمعة

١. Blob Storage مقابل Azure Files

المعيار	Blob Storage	Azure Files
بنية التخزين	مسطحة (Flat Namespace)	هرمية حقيقية (مجلدات)
طريقة الوصول	REST API / SDK / URL	SMB / NFS / REST API
الاستخدام الأمثل	بيانات غير منظمة كبيرة الحجم	مشاركة ملفات بين عدة مستخدمين

٢. Queue Storage مقابل Blob Storage

المعيار	Blob Storage	Queue Storage
الغرض	تخزين ملفات كبيرة	تبادل رسائل صغيرة
حجم الكائن	حتى عدة تيرابايت للملف الواحد	حتى ٦٤ كيلوبايت للرسالة الواحدة
مثال استخدام	تخزين فيديو	إشعار معالجة طلب

٣. Table Storage مقابل Blob Storage

المعيار	Blob Storage	Table Storage
نوع البيانات	غير منظمة (ملفات)	شبه منظمة (Key-Value)
الاستعلام	لا يدعم استعلامات معقدة	يدعم استعلامات عبر المفتاح
مثال استخدام	صورة منتج	سجل بيانات مستشعر IoT

٤. Storage Account مقابل Blob Container

المعيار	Storage Account	Blob Container
المستوى	المستوى الأعلى (الحساب الكامل)	مستوى فرعي داخل خدمة Blob فقط
يحتوي على	Blob, Files, Queue, Table	مجموعة من الـ Blobs فقط
الاستقلالية	لا يعتمد على أي كيان آخر	لا يمكن أن يوجد بدون Storage Account

٥. Public Access مقابل Private Access

المعيار	Public Access	Private Access
من يمكنه الوصول	أي شخص لديه الرابط	فقط من يملك تصريحًا صالحًا (Key/SAS)
المخاطر الأمنية	مرتفعة عند البيانات الحساسة	منخفضة نسبيًا
الاستخدام الموصى به	محتوى عام (صور موقع عام)	بيانات العملاء والملفات الحساسة

سادساً: الأخطاء والمشاكل الشائعة

◆ عدم القدرة على الوصول إلى Blob

السبب: غالباً بسبب أن الـ Container مضبوط على Private ولا يوجد رابط SAS صالح.

الحل: توليد رابط SAS بصلاحيات مناسبة أو تفعيل الوصول العام إذا كانت البيانات غير حساسة.

◆ ظهور رسالة (403) Access Denied

السبب: محاولة الوصول المباشر لملف خاص دون بيانات اعتماد صحيحة، أو انتهاء صلاحية SAS.

الحل: التحقق من صلاحية الرابط وتاريخ انتهائه، أو استخدام Access Key/Entra ID بشكل صحيح.

◆ إعداد Public Access بشكل خاطئ

السبب: اختيار مستوى وصول عام أثناء إنشاء Container دون إدراك الأثر الأمني.

الحل: مراجعة إعدادات الوصول فور الإنشاء وتصحيحها إلى Private عند الحاجة.

◆ استخدام Access Key بطريقة غير آمنة

السبب: تضمين المفتاح مباشرة في الكود المصدري أو رفعه إلى مستودع عام.

الحل: استخدام SAS أو Microsoft Entra ID بدلاً من المفتاح الكامل، وتخزين الأسرار في Azure Key Vault.

◆ اختيار Redundancy غير مناسب

السبب: اختيار LRS لبيانات حرجة جداً، أو اختيار GZRS لبيانات غير مهمة (تكلفة زائدة).

الحل: تقييم أهمية البيانات فعلياً قبل الاختيار، وربط القرار بخطة استمرارية الأعمال.

◆ ارتفاع تكلفة التخزين

السبب: إبقاء كل البيانات في طبقة Hot رغم قلة الوصول إليها، أو تراكم بيانات قديمة غير مستخدمة.

الحل: تفعيل Lifecycle Management لنقل البيانات القديمة إلى Cool/Archive أو حذفها.

◆ عدم معرفة الفرق بين Storage Account و Container

السبب: الخلط بين المستويين الإداريين المختلفين.

الحل: الرجوع إلى الرسم الهرمي الموضح في هذا التقرير لتثبيت الفهم.

◆ عدم القدرة على رفع ملف

السبب: مشكلة في اتصال الإنترنت، أو تجاوز حجم الملف الأقصى المسموح به لنوع Blob المستخدم.

الحل: التحقق من الاتصال، واستخدام Block Blob للملفات الكبيرة مع الرفع على دفعات (Chunks).

◆ مشاكل الصلاحيات (Permissions)

السبب: المستخدم أو التطبيق لا يملك الدور (Role) المناسب في Azure RBAC.

الحل: إسناد الدور الصحيح مثل Storage Blob Data Contributor عبر Access Control (IAM).

سابعًا: أفضل الممارسات (Best Practices)

- حماية البيانات: تفعيل التشفير الافتراضي (مفعّل تلقائيًا) وعدم تعطيله أبدًا.
- استخدام Private Access كأعداد افتراضي، وعدم فتح الوصول العام إلا عند الضرورة القصوى.
- إدارة الصلاحيات عبر Azure RBAC و Microsoft Entra ID بدلاً من الاعتماد الكلي على Access Keys.
- استخدام Shared Access Signature (SAS) بمدة صلاحية قصيرة وصلاحيات محدودة عند الحاجة لمشاركة مؤقتة.
- تفعيل Require secure transfer لفرض الاتصال عبر HTTPS فقط.
- اختيار نوع Redundancy بما يتناسب مع أهمية البيانات الفعلية لا بشكل عشوائي.
- تقليل التكلفة عبر تصنيف البيانات إلى طبقات (Hot/Cool/Archive) وتفعيل سياسات دورة الحياة.
- تنظيم الـ Containers بأسماء واضحة ومنطقية تسهل الإدارة والمراقبة لاحقًا.
- إعداد نسخ احتياطية دورية (Backup) وتفعيل خاصية Soft Delete للحماية من الحذف العرضي.
- مراقبة استخدام التخزين والتكلفة بشكل دوري عبر Azure Monitor و Azure Cost Management.

ثامناً: أسئلة وأجوبة متوقعة للمناقشة

١. ما هو Azure Storage Account؟

الإجابة: هو الكيان الإداري الرئيسي في Azure الذي يحتضن جميع خدمات التخزين (Blob، Files، Queue، Table) ويمنحها نقطة وصول موحدة عبر الإنترنت.

٢. ما الفرق بين Storage Account وContainer؟

الإجابة: Storage Account هو المستوى الأعلى ويضم كل الخدمات، بينما Container مستوى فرعي داخل خدمة Blob فقط لتنظيم الملفات، ولا يمكن أن يوجد بدون Storage Account.

٣. ما هو Blob؟

الإجابة: هو أي ملف يُرفع إلى خدمة Blob Storage، ويصبح كائنًا له خصائص وصلاحيات وصول خاصة به، ويُخزَّن داخل Container.

٤. ما الفرق بين Azure Files وBlob Storage؟

الإجابة: Blob مخصص لتخزين البيانات غير المنظمة بشكل مسطح ويُصل إليه عبر REST/URL، بينما Azure Files يوفر مشاركات ملفات هرمية يمكن الوصول إليها كمجلد شبكة عبر SMB/NFS.

٥. لماذا نستخدم Redundancy؟

الإجابة: لحماية البيانات من فقدان عند حدوث أعطال بالاحتفاظ بأكثر من نسخة منها في أماكن مختلفة، مما يضمن استمرارية توفر البيانات.

٦. ما الفرق بين Public وPrivate Access؟

الإجابة: Public يسمح لأي شخص بالوصول عبر الرابط المباشر، بينما Private يمنع الوصول المباشر ويتطلب مصادقة أو رابط SAS مؤقت.

٧. ما المخاطر الأمنية لـ Public Access؟

الإجابة: تعريض البيانات الحساسة لخطر الوصول من جهات غير مصرح لها، بما يشمل الزحف الآلي، مما قد يؤدي إلى تسريب بيانات.

٨. كيف نحمي البيانات في Azure Storage؟

الإجابة: عبر التشفير التلقائي، فرض HTTPS، ضبط الوصول على Private، استخدام SAS، وإدارة الصلاحيات عبر RBAC وMicrosoft Entra ID.

٩. لماذا نستخدم SAS؟

الإجابة: لمنح وصول مؤقت ومحدود الصلاحيات لجهة معينة دون الحاجة لمشاركة مفتاح الحساب الكامل (Access Key).

١٠. ما العوامل التي تحدد تكلفة Azure Storage؟

الإجابة: كمية البيانات، عدد عمليات القراءة/الكتابة، نوع طبقة التخزين، نوع Redundancy، وحجم البيانات المنقولة خارج المنطقة.

١١. ما الفرق بين Block Blob وAppend Blob وPage Blob؟

الإجابة: Block Blob للملفات العامة، Append Blob للبيانات المضافة تباغًا مثل السجلات، وPage Blob للقراءة/الكتابة العشوائية مثل أقراص الأجهزة الافتراضية.

١٢. ما هو File Share؟

الإجابة: مساحة تخزين داخل خدمة Azure Files تُشبه مجلدًا مشتركًا على شبكة يمكن لعدة مستخدمين الوصول إليه في آن واحد.

١٣. متى نستخدم Queue Storage؟

الإجابة: عندما نحتاج لفصل مكونات النظام عن بعضها والسماح بمعالجة غير متزامنة للمهام عبر تبادل رسائل صغيرة.

١٤. ما هو Table Storage ومتى نستخدمه؟

الإجابة: قاعدة بيانات NoSQL بسيطة لتخزين بيانات شبه منظمة بكميات كبيرة، تُستخدم عند الحاجة لاستعلامات سريعة عبر مفتاح دون علاقات معقدة.

١٥. ما الفرق بين LRS وGRS؟

الإجابة: LRS ينسخ البيانات ٣ مرات داخل نفس مركز البيانات فقط، بينما GRS ينسخها محليًا ثم جغرافيًا إلى منطقة بعيدة لحماية إضافية من الكوارث.

١٦. ما هو Microsoft Entra ID وما علاقته بالتخزين؟

الإجابة: نظام إدارة هوية مركزي (سابقًا Azure AD) يتيح مصادقة وتفويضًا أدق للمستخدمين والتطبيقات بدلاً من الاعتماد فقط على مفاتيح الحساب.

١٧. كيف نقلل تكلفة التخزين؟

الإجابة: باستخدام طبقات التخزين المناسبة (Cool/Archive)، اختيار Redundancy مناسب، وتفعيل سياسات دورة حياة البيانات لحذف أو نقل البيانات القديمة تلقائيًا.

١٨. هل يمكن الوصول إلى Blob بدون رابط SAS إذا كان الـ Container خاصًا؟
الإجابة: لا، لن يُسمح بالوصول المباشر عبر الرابط، ويجب استخدام SAS أو بيانات اعتماد صالحة (Access Key أو Entra ID).

١٩. ما اسم الحساب الفريد المطلوب عند إنشاء Storage Account؟
الإجابة: يجب أن يكون اسمًا فريدًا عالميًا (Globally Unique) بحروف صغيرة وأرقام فقط دون مسافات أو رموز خاصة.

٢٠. ما الفرق بين Standard وPremium في إعداد Performance؟
الإجابة: Standard مناسب لمعظم الاستخدامات العامة وأرخص تكلفة، بينما Premium مخصص للأداء العالي جدًا (زمن استجابة أقل) بتكلفة أعلى.

تاسعًا: الخاتمة

تناول هذا التقرير مفهوم Azure Storage Account بوصفه الكيان الإداري الأساسي في منصة Microsoft Azure الذي يحتضن خدمات التخزين الأربعة الرئيسية: Blob Storage لتخزين البيانات غير المنظمة، و Azure Files لمشاركة الملفات كمجلد شبكة، و Queue Storage لتبادل الرسائل بين مكونات الأنظمة، و Table Storage لتخزين البيانات شبه المنظمة بسرعة عالية.

كما وضّح التقرير الفرق الجوهرية بين Storage Account و Blob Container من خلال العلاقة الهرمية بينهما، وأهمية مفهوم Redundancy في حماية البيانات من فقدان عبر أنواعه الأربعة (LRS و ZRS و GRS و GZRS)، والفرق بين Public Access و Private Access وأثره المباشر على أمان البيانات، إضافة إلى أهم الاعتبارات الأمنية (المصادقة، التفويض، التشفير، Microsoft Entra ID، SAS) والعوامل المؤثرة في التكلفة.

أما من الناحية العملية، فقد تم تنفيذ سيناريو متكامل بدءًا من إنشاء Resource Group، مرورًا بإنشاء Storage Account و Blob Container ورفع الملفات إليه، وصولًا إلى إنشاء File Share ومراجعة إعدادات Access و Redundancy داخل Azure Portal. وقد أظهر هذا التطبيق العملي كيف تتربط جميع المفاهيم النظرية معًا في سياق واقعي قابل للتنفيذ الفعلي.

الخلاصة: فهم Azure Storage Account بعمق — نظريًا وعمليًا — يُعد أساسًا ضروريًا لأي مهتم بالحوسبة السحابية، نظرًا لكونه نقطة الانطلاق لأي مشروع يحتاج إلى تخزين بيانات موثوق وآمن وقابل للتوسع.

عاشراً: قائمة الأشكال والصور

- الشكل (١): إنشاء Resource Group
- الشكل (٢): إنشاء Storage Account وإعداداته
- الشكل (٣): إنشاء Blob Container وتحديد مستوى الوصول
- الشكل (٤): رفع ملف إلى Blob Container
- الشكل (٥): خصائص الملف ورابط الوصول (URL) وتوليد SAS
- الشكل (٦): إنشاء File Share ورفع ملف إليه
- الشكل (٧): إعدادات Redundancy و Access داخل Storage Account

حادي عشر: المراجع

Microsoft Learn — Introduction to Azure Storage

<https://learn.microsoft.com/azure/storage/common/storage-introduction>

Microsoft Learn — Blob Storage Overview

<https://learn.microsoft.com/azure/storage/blobs/storage-blobs-overview>

Microsoft Learn — Azure Files Overview

<https://learn.microsoft.com/azure/storage/files/storage-files-introduction>

Microsoft Learn — Queue Storage Overview

<https://learn.microsoft.com/azure/storage/queues/storage-queues-introduction>

Microsoft Learn — Table Storage Overview

<https://learn.microsoft.com/azure/storage/tables/table-storage-overview>

Microsoft Learn — Azure Storage Redundancy

<https://learn.microsoft.com/azure/storage/common/storage-redundancy>

Microsoft Learn — Shared Access Signatures (SAS)

<https://learn.microsoft.com/azure/storage/common/storage-sas-overview>

Microsoft Learn — Manage Anonymous Read Access

<https://learn.microsoft.com/azure/storage/blobs/anonymous-read-access-configure>